

Inference Optimisation for Large Models

Distillation, Quantization, and TurboQuant

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

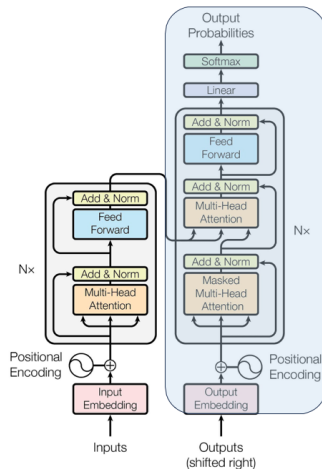
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

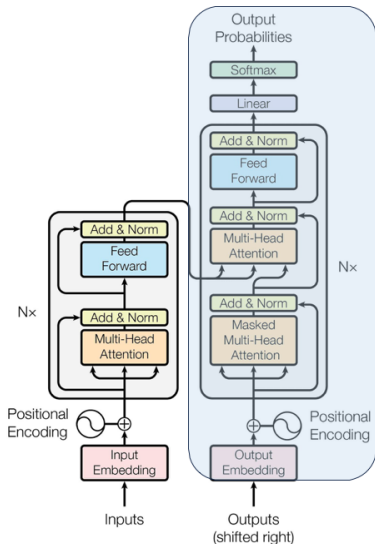
May 27, 2026

1. LLM inference: autoregressive decoding, prefill vs. decode phases
2. Inference optimisation: KV caching, memory scaling, and other serving levers
3. Knowledge distillation: compression motivation and teacher–student architecture
4. Distillation training: soft targets, temperature, loss, and deployment scenarios
5. Quantization: formats, granularity, weights/activations/KV cache, QAT and PTQ
6. TurboQuant: two-stage vector quantization (PolarQuant + QJL) for KV compression
7. Summary and references

- ▶ Most popular decoder-only LLMs (e.g., GPT-3) are pretrained for **causal language modeling**: predicting the next token in a sequence.
- ▶ At inference, LLMs receive a series of **tokens** as input and generate the following tokens **autoregressively** until a stopping criterion is met (e.g., token limit, stop word, or special <end> token).
- ▶ **Tokens** are about 4 English characters each.
- ▶ The inference process has **two phases**: **prefill** (process prompt in parallel) and **decode** (step-by-step token generation).

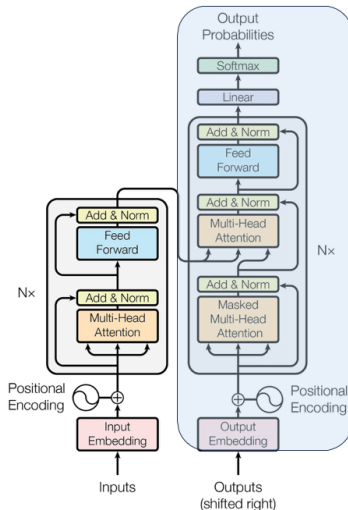


- ▶ **Prefill phase (input processing):**
 - All input tokens are processed together to compute **keys and values** (attention states) for the prompt.
 - The entire input sequence is known in advance, enabling a highly parallel **matrix-matrix** computation that fully leverages the GPU's throughput.
- ▶ **Token throughput caveat:**
 - Different LLMs may use different **tokenizers**—so "tokens per second" is not always comparable across models, as token-to-character mappings vary.



► Decode phase (generating outputs):

- Output tokens are generated **one at a time**, each conditioned on all previously generated tokens.
- Computation now becomes a **matrix-vector** operation—less parallelizable, and typically **memory-bound**: moving weights/activations dominates speed, not raw compute.
- Optimising decode is crucial: efficient attention, key/value caching, and memory management are ongoing research topics.



- ▶ **Training** is often one-off; **inference** is repeated at scale (cost, latency, energy).
- ▶ Large language models are **memory-bound** and **autoregressive**: each new token revisits all previous tokens.
- ▶ Goals: lower **latency** (interactive UX), higher **throughput** (serving many users), smaller **memory** (long contexts, edge).

- ▶ In **decode**, the model emits **one token per step**, but that token still depends on the **key and value** tensors for **all** earlier positions.
- ▶ Those positions include the **prompt** (K/V from **prefill**) plus any **new** K/V computed up to the current step.
- ▶ **KV caching** keeps these tensors in GPU memory so they are **not recomputed** at every step; each iteration **appends** the newly computed K/V to the cache for the next step.
- ▶ In many stacks there is **one KV cache per transformer layer**.

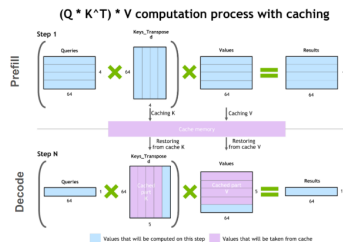


Figure: illustration of the key-value caching mechanism. Reference: Verma & Vaidya, "Mastering LLM Techniques: Inference Optimization," NVIDIA Technical Blog (2023).

- ▶ With batching, each request needs its own KV cache, which can become a bottleneck even before model weights do.
- ▶ Cost increases in proportion to batch size and sequence length, limiting throughput and ability to handle long inputs. This drives use of techniques like MQA/GQA, paging (PagedAttention), and KV quantization.

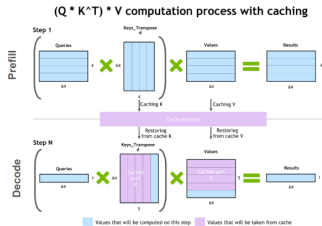


Figure: illustration of the key-value caching mechanism. Reference: Verma & Vaidya, “Mastering LLM Techniques: Inference Optimization,” NVIDIA Technical Blog (2023).

- ▶ **Kernel fusion & IO-aware attention** (e.g. FlashAttention-style): reduce HBM traffic.
- ▶ **Batching / continuous batching**: pack requests to improve GPU utilisation.
- ▶ **Weight sharing / low-rank adapters (LoRA)**: smaller footprints where applicable.
- ▶ **Approximate attention** (long contexts): trade accuracy for sub-quadratic cost.

Model Optimisation Techniques



Problem:

- ▶ During **training**, a model does not have to operate in real time and does not necessarily face restrictions on computational resources. The primary goal is to extract as much structure from the given data as possible.
- ▶ But latency and resource consumption do become of concern if the model is to be deployed for **inference**.

So what? We must develop ways to compress models for inference.

Idea:

- ▶ In 2006, Bucilă et al. showed that it was possible to transfer knowledge from a large trained model (or ensemble of models) to a smaller model for deployment by **training it to mimic the larger model's output**.
- ▶ In 2014 Hinton et al. generalized the process and gave the name **Distillation**.

Main idea of distillation is that **training and inference are 2 different tasks**; thus a **different model should be used**.

Assumption: If we can achieve similar convergence using a **smaller network**, then the convergence space of the Teacher Network should **overlap** with the solution space of the Student Network.



Teacher/Student convergence regions. Harvard AC215, L11.

While training, our confidence in the labels must be high.

Otherwise the data would not be present in the training set.

For example:

This is a Dog

one-hot		0
		0
		1
		0

→ Dog



While training, our confidence in the labels must be high.

Otherwise the data would not be present in the training set.

For example:

This is also a Dog

one-hot		0	→ Dog
		0	
		1	
		0	



While training, our confidence in the labels must be high.

Otherwise the data would not be present in the training set.

For example:

This is a Cat					
one-hot	<table><tr><td>0</td></tr><tr><td>0</td></tr><tr><td>1</td></tr><tr><td>0</td></tr></table>	0	0	1	0
0					
0					
1					
0					

→ Dog



What about this example?

This is a ...

$$\begin{pmatrix} 0 \\ 0.5 \\ 0.5 \\ 0 \end{pmatrix}$$



In the real world, the distinction is never that clear.

Ideally, the **training labels must be soft**, to learn about the nuanced relations across the elements of the dataset.



The teacher shows the student the **relations** between the different classes, based on what it **learned** during its training stage.

From a dataset containing the original categories, the teaching outputs the **normalized probabilities** for each example.

	cow	dog	cat
Original labels	0	1	0
Probabilities	10^{-6}	0.9	0.1

Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Dark Knowledge*

Distillation: Teacher – Student (Softening with Temperature)

To enhance the student's learning, the teacher **softens** its output z even more.

To do this, introduces the softmax function with temperature T :

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

“Soft” targets allow the student to learn the teacher’s implicit knowledge about class similarities.

	cow	dog	cat
Original labels	0	1	0
Softmax probabilities	10^{-6}	0.9	0.1
Softened probabilities 0.005	0.05	0.3	0.2

Adapted from:
Geoffrey Hinton, Oriol Vinyals & Jeff Dean,
Dark Knowledge

Distillation: Teacher – Student (Softening with Temperature)

A high value of T will dilute the information contained in the probabilities.

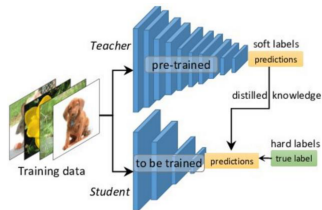
- ▶ The standard softmax is recovered for $T=1$.
- ▶ For $T>1$, the probabilities are softened. Typical values range between 2 and 5.
- ▶ For $T<1$, it approaches to the original one-hot labels.

	cow	dog	cat
Original labels	0	1	0
Softmax probabilities	10^{-6}	0.9	0.1
Softened probabilities 0.005	0.05	0.3	0.2

Adapted from:
Geoffrey Hinton, Oriol Vinyals & Jeff Dean,

Dark Knowledge

- ▶ The student is trained to both:
 - Solve the original task (matching the hard/true labels).
 - Replicate the softened probabilities (soft labels) from the teacher.
- ▶ This allows the student to learn not only the correct answer, but also the teacher's representation of class similarities.



Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Dark Knowledge*

Scenario 1:

Train on the same data as the teacher model.

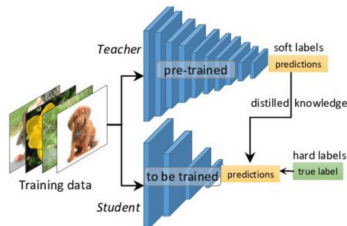
(DistilBERT was trained on this example.)

Scenario 2:

Train on a reduced dataset. This is usually done when you have the teacher model but not the original dataset or resources.

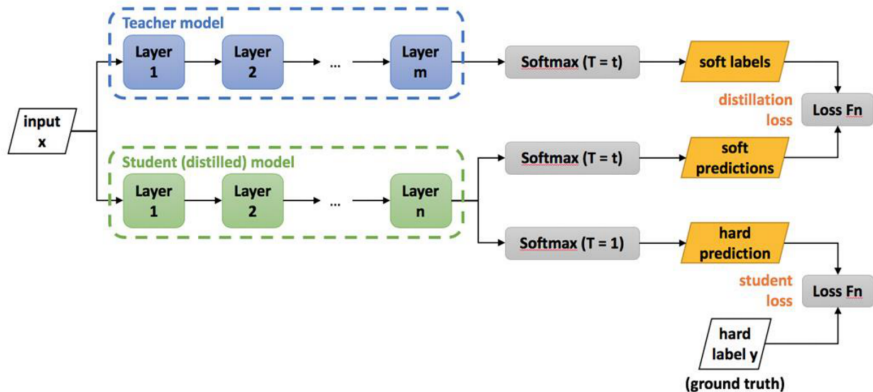
It minimizes the sum of two different cross-entropy functions:

- ▶ One involving the original hard labels obtained using a softmax with $T = 1$.
- ▶ One involving the softened probabilities with $T > 1$.



Geoffrey Hinton, Oriol Vinyals & Jeff Dean,
Dark Knowledge

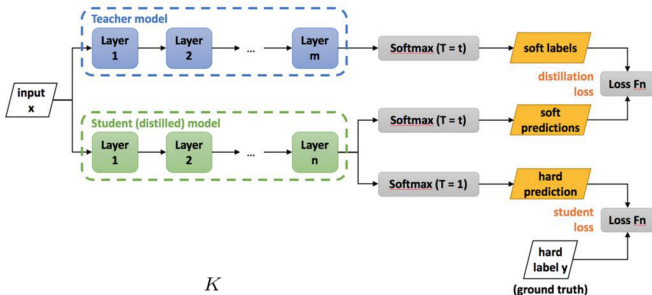
Distillation: Teacher–Student



Reference: Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Dark Knowledge*.

$$L = \lambda L_{\text{student}} + (1 - \lambda) L_{\text{distillation}}$$

Distillation: Teacher Student Training



$$L_{Distillation} = - \sum_k^K p_T^k(x, T) \log(q_S^k(x, T))$$

33

$$L_{\text{Distillation}} = - \sum_{k=1}^K p_T^k(x, T) \log(q_S^k(x, T))$$

In our example, for the example n in the dataset:

$$p_T(x, T) = [0.05, 0.3, 0.2, 0.005]$$

And our predictions

$$q_T(x, T) = [0.1, 0.45, 0.30, 0.15]$$

The resulting loss value for this example is:

$$-\left[0.05 \cdot \log(0.1) + 0.3 \cdot \log(0.45) + 0.2 \cdot \log(0.30) + 0.005 \cdot \log(0.15)\right]$$

Quantization

Quantization is a **key** compression technique for **LLMs** and **large transformers** to fit on **constrained hardware**.

- ▶ **Goal:** run larger models on **constrained** hardware with acceptable accuracy.
- ▶ Lowering precision on **weights and/or activations** (e.g. FP32/FP16 → FP8) cuts **memory**, speeds **inference**, and reduces **energy**—at some accuracy cost; the right trade-off is use-case dependent.

- ▶ **Weights:** straightforward win on footprint; Llama 2 7B at FP16/BF16 $\approx$ **14 GB** $\rightarrow$ FP8 weights $\approx$ **7 GB** (blog numbers).
- ▶ **Activations:** intermediate layer outputs; quantizing compute can exploit **tensor cores** at lower bit width for more throughput.
- ▶ **KV cache** (decoder-only): grows with context, layers, and heads—often **several GB** at long context (e.g. 4096 tokens) alongside weights.
- ▶ **Deployment patterns:** weight-only INT8/INT4 for maximum weight memory savings (activations may stay higher precision); **KV cache quantization** when long context dominates.

- ▶ Floating point formats use their bits to store three parts: **sign** (positive/negative), **exponent** (size/range), and **mantissa** (precision of the value). For example, FP8 E4M3 uses 4 bits for the exponent and 3 bits for the mantissa.
- ▶ These numbers are represented as $x = (-1)^{\text{sign}} \cdot 2^{\text{exponent}} \cdot \text{mantissa}$.
- ▶ Typical formats: FP32, FP16, BF16, FP8, and FP4. Main difference is how much range and precision you get—higher precision means more accurate numbers, but also more memory and compute.



Figure: FP16, BF16, FP8, FP4 layouts.

- ▶ **Per-tensor / per-layer:** one (s, z) for the whole tensor—simplest, smallest metadata, can mis-handle heterogeneous ranges.
- ▶ **Per-channel:** separate parameters per channel (e.g. conv channels)—isolates **outliers** to their channel.
- ▶ **Per-block / per-group:** parameters per sub-block—finer control when mass varies inside a tensor.

Per-Tensor:



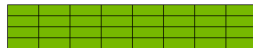
All elements share the same scale and zero-point

Per-Channel:



Each channel (horizontal strip) has its own scale and zero-point

Per-Group:



Each group (small block) has its own scale and zero-point

Figure: per-tensor, per-channel, per-block.

- ▶ Bake quantization into training: forward/backward see **fake-quant** (quantize then dequantize) while master weights stay high precision for optimiser steps.
- ▶ Weights adapt to **rounding/clipping** error; fake-quant modules often **frozen** while weights are fine-tuned.

PTQ: Weight-Only and Weight+Activation Quantization (Overview)

- ▶ **Weight-only quantization:** Quantize the model's trained weights to lower precision using a calculated scale (with or without a zero-point). Since weights are fixed after training, no input data is needed.
- ▶ **Weights and activations quantization:** Requires representative input data to measure activation statistics (by running the model), which are then used to determine quantization parameters (scale and zero-point) through a process called calibration.

- ▶ **Static quantization:** Calibration uses a dataset to determine quantization parameters once; these are kept fixed for all future inference.
- ▶ **Dynamic quantization:** Quantization parameters are computed on the fly during inference for each input, so no calibration set is needed but more runtime overhead is incurred.

TurboQuant

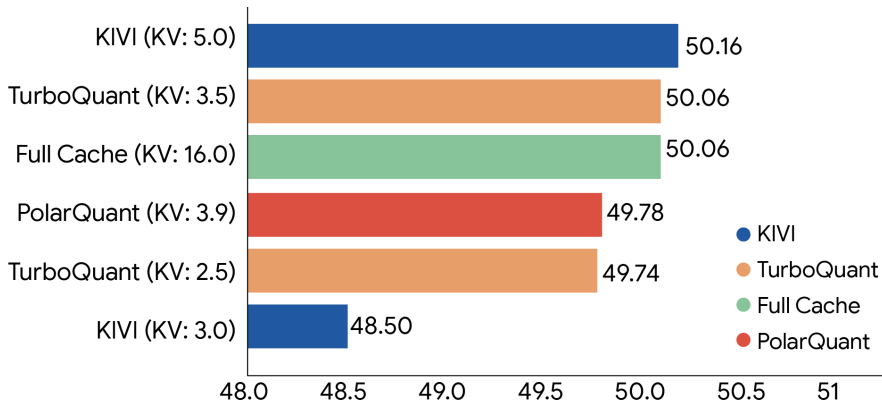
- ▶ **Vector quantization (VQ):** compress high-dimensional vectors while controlling **distortion** of their geometry.
 - ▶ Enhances vector search and KV cache compression.
- ▶ Prior practical schemes often fall **short of optimal** distortion–rate scaling.
- ▶ TurboQuant is a compression method that reduces model size with zero accuracy loss, addressing the constraints of long contexts and agentic workflows.
- ▶ Applies to any model at inference time with no offline preparation or calibration data.

► TurboQuant works in **two main stages**:

- **Stage 1: High-quality compression with PolarQuant.** TurboQuant first randomly rotates the data vectors, simplifying their geometry. This lets us apply a strong, standard quantizer (like those used in audio or image compression) to each dimension, using most of the available bits to efficiently capture the main “strength” and “direction” of the original vector.
- **Stage 2: Eliminating hidden errors with QJL.** After the main compression, a tiny bit of error remains. TurboQuant uses just 1 bit and the **QJL (Quantized Johnson–Lindenstrauss)** algorithm to compress this residual, acting like a mathematical error-checker that eliminates bias and ensures precise attention scores.

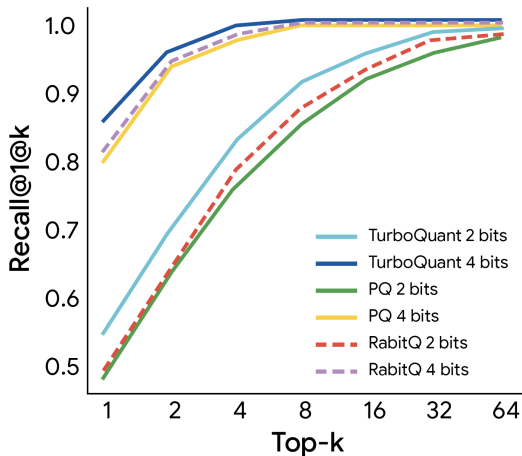
Zandieh & Mirrokni, “TurboQuant: Redefining AI efficiency with extreme compression,” Google Research Blog (2026). [Article](#).

TurboQuant: Experiments and Results



Aggregated scores across LongBench-style tasks (QA, code, summarization) for TurboQuant, PolarQuant, and KIVI baselines; bit-widths in brackets. Reference: Google Research Blog.

TurboQuant: Experiments and Results



1@k recall on GloVe ($d=200$): TurboQuant vs. PQ / RabbitQ-style baselines. Reference: Google Research Blog.

- ▶ **Inference optimisation lowers latency, increases throughput, and reduces memory.**
- ▶ **KV caching** avoids recomputing past keys/values but grows memory with layers, batch, and length—motivating KV quantization, grouped attention, paging, etc.
- ▶ **Distillation:** train a compact **student** to mimic a **teacher** for deployment; **soft targets** at raised **temperature** carry knowledge beyond one-hot labels.
- ▶ **Quantization:** shrink **weights**, **activations**, and the **KV cache** via lower-bit formats.
- ▶ **TurboQuant:** a compression method that reduces model size with zero accuracy loss—addressing long-context and agentic workflow constraints—and applies to any model at inference time with no offline preparation or calibration data.

1. Vaswani et al., “Attention Is All You Need,” NeurIPS 2017. [arXiv:1706.03762](#)
2. Hinton, Vinyals, Dean, “Distilling the Knowledge in a Neural Network,” NIPS Deep Learning Workshop 2015. [arXiv:1503.02531](#)
3. Zelikman et al., “STaR: Bootstrapping Reasoning With Reasoning,” NeurIPS 2022. [arXiv:2203.14465](#)
4. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” CVPR 2018. [arXiv:1712.05877](#)
5. Leviathan, Kalman, Matias, “Fast Inference from Transformers via Speculative Decoding,” ICML 2023. [arXiv:2302.01318](#)
6. Verma, “Mastering LLM Techniques: Inference Optimization,” NVIDIA Developer Blog, 17 Nov. 2023 (updated 15 Aug. 2024). [Article](#)
7. Wang, Spindler, “Model Quantization: Concepts, Methods, and Why It Matters,” NVIDIA Developer Blog, 24 Nov. 2025. [Article](#)
8. Zandieh, Daliri, Hadian, Mirrokni, “TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate,” 2025. [arXiv:2504.19874](#)
9. Zandieh, Mirrokni, “TurboQuant: Redefining AI efficiency with extreme compression,” Google Research Blog, 24 Mar. 2026. [Article](#)

10. Khattab et al., *LLM Class: Lecture 1—Introduction and Course Overview* (slide deck). [PDF](#)
11. Harvard University IACS, *CS AC215—Advanced Practical Data Science* (Spring 2024), Lecture 11: *Compression techniques* (slides 8–30 used for distillation figures). [PDF](#)
12. CMU 15-779 (*Lecture 17: LLM fine-tuning* — slide deck). [PDF](#)